

Nuclei — Study Guide

Nuclei (by ProjectDiscovery) is a fast, modern, **YAML-template-based vulnerability scanner** — the tool that's largely taken over the "quick, broad vulnerability sweep" role that Nikto used to own alone, thanks to its speed, extensibility, and a massive community-maintained template library that gets new CVE checks added within hours of public disclosure.

1. What Makes Nuclei Different

Unlike Nikto (fixed built-in checks) or Nessus (proprietary plugin format), Nuclei's checks are **YAML templates** — human-readable, easy to write, and openly shared. This means:

- New CVEs get community-written templates almost immediately after public disclosure
- You can write your own custom templates in minutes for a specific finding pattern
- Templates cover far more than just web servers — CVEs, misconfigurations, exposed panels, default logins, DNS issues, cloud misconfig, and more

```
Nikto    → fixed checks, web-server-focused
Nuclei   → community-driven templates, broad scope, extremely fast (built in Go, highly concurrent)
```

2. Installation

```
# Go-based install (Nuclei is written in Go)
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest

# Or via apt on Kali (often slightly behind latest release)
sudo apt install nuclei
```

First run downloads the template repository:

```
nuclei -update-templates
```

3. Basic Usage

```
nuclei -u https://target.com
nuclei -l targets.txt           # scan a list of targets
nuclei -u https://target.com -severity critical,high # filter by severity
```

4. Key Flags

Flag	Meaning
<code>-u</code>	Single target URL
<code>-l</code>	List of targets from a file
<code>-t</code>	Specify specific template(s)/directory
<code>-severity</code>	Filter by severity (info, low, medium, high, critical)
<code>-tags</code>	Run templates matching specific tags (e.g., <code>cve</code> , <code>exposure</code> , <code>misconfig</code>)
<code>-o</code>	Output file
<code>-json</code>	JSON output (scriptable)
<code>-rate-limit</code>	Requests per second (throttle to be less noisy)
<code>-c</code>	Concurrency (threads)
<code>-etags</code>	Exclude specific tags

5. Templates — The Core Concept

Templates live in `~/nuclei-templates/` after updating, organized by category:

```
cves/           # specific CVE checks, organized by year
exposures/      # exposed panels, config files, API keys
misconfiguration/ # default installs, debug modes enabled
```

```
vulnerabilities/          # generic vuln classes (SQLi,
XSS patterns)
technologies/            # fingerprinting - what's ru
nning
default-logins/          # known default credential
checks
```

Running by category:

```
nuclei -u target -t cves/
nuclei -u target -t exposures/
nuclei -u target -tags cve,rce
```

A simple template example (to understand the format):

```
id: exposed-env-file

info:
  name: Exposed .env File
  severity: high
  tags: exposure,config

http:
  - method: GET
    path:
      - "{{BaseURL}}/.env"
    matchers:
      - type: word
        words:
          - "DB_PASSWORD"
        part: body
```

This directly mirrors the `.env` exposure dorks from your Google Dorking guide — Nuclei is essentially automating the same detection logic as an active check instead of a passive search query.

6. Writing Your Own Template (Genuinely Worth Doing)

Since templates are just YAML, writing a custom one is approachable even early in your learning:

```
id: custom-admin-panel-check

info:
  name: Custom Admin Panel Finder
  severity: info
  tags: custom,panel

http:
  - method: GET
    path:
      - "{{BaseURL}}/admin"
      - "{{BaseURL}}/administrator"
    matchers:
      - type: status
        status:
          - 200
```

```
nuclei -u target -t my-custom-template.yaml
```

Writing 2-3 custom templates targeting patterns specific to your own testing needs is a strong, demonstrable portfolio addition — "wrote custom Nuclei templates" is a concrete, specific line for a resume/GitHub README.

7. Combining Nuclei With Recon Tools (How It's Actually Used at Scale)

Nuclei is designed to chain with other ProjectDiscovery tools (and your earlier subdomain enum tools):

```
subfinder -d example.com | httpx -silent | nuclei -t cves/ -s
everity critical,high
```

This one-liner: finds subdomains → checks which are alive → runs critical/high CVE templates against every live host — a genuinely production-grade recon-

to-vuln-scan pipeline in a single piped command, and a great example to reference if asked about automation experience.

8. Practical Example — Full Scan

```
nuclei -u https://target.com -t cves/,exposures/,misconfiguration/ -severity medium,high,critical -o nuclei_results.txt
```

Reading output:

```
[CVE-2021-44228] [critical] [http] https://target.com/api/login
[exposed-env-file] [high] [http] https://target.com/.env
[default-login] [medium] [http] https://target.com/admin
```

Clean, one-line-per-finding format — easy to grep/parse, and directly feeds into a report or into further Python processing (again, exactly the kind of API/data-parsing pattern your SmartScan project already uses with NVD).

9. Rate Limiting and Being a Good Citizen

```
nuclei -u target -rate-limit10 -c5 # slower, gentler scan
```

Nuclei's default speed can be aggressive — always throttle appropriately in real engagements, both to avoid overwhelming a target and to stay within authorized scope/rules of engagement.

10. Kill Chain / ATT&CK Mapping

Framework	Mapping
Cyber Kill Chain	Reconnaissance
MITRE ATT&CK	T1595.002 (Active Scanning: Vulnerability Scanning)

Quick Reference Cheat Sheet

```
nuclei -update-templates
nuclei -u https://target.com
nuclei -l targets.txt -severity critical,high
nuclei -u target -t cves/,exposures/
nuclei -u target -tags rce,cve -o results.txt
subfinder -d example.com | httpx -silent | nuclei -t cves/
```